

**Diseño e Implementación de un Agente en Espacio de
Usuario**
**para la Gestión Dinámica de Frecuencia (DVFS) en
Sistemas**
**Heterogéneos mediante Modelos Ligeros de Machine
Learning**

Yeison Adrián Cáceres Torres

Ricardo Andrés Pérez Porras

Trabajo de Grado para Optar el Título de Ingeniero de Sistemas

Director

Gilberto Javier Díaz Toro

Doctor en Ingeniería

Universidad Industrial de Santander

Facultad de Ingenierías Físico-Mecánicas

Escuela de Ingeniería de Sistemas e Informática

Bucaramanga

2026

Agradecimientos

Resumen

Título: Diseño e Implementación de un Agente en Espacio de Usuario para la Gestión Dinámica de Frecuencia (DVFS) en Sistemas Heterogéneos mediante Modelos Ligeros de Machine Learning.

Autores: Yeison Adrián Cáceres Torres; Ricardo Andrés Pérez Porras.

Palabras Clave: DVFS; eficiencia energética; computación de alto rendimiento; modelo Roofline; telemetría microarquitectónica; aprendizaje automático supervisado.

Descripción:

El consumo energético constituye hoy una restricción estructural para el escalamiento de la computación de alto rendimiento (HPC), particularmente en nodos heterogéneos que integran CPU y aceleradores gráficos. El Escalado Dinámico de Voltaje y Frecuencia (DVFS) es el principal mecanismo de control disponible a nivel de hardware, pero su aprovechamiento efectivo depende de decidir, durante la ejecución, cuál punto operativo conviene según el comportamiento de la carga. Los gobernadores nativos del sistema operativo deciden a partir de la utilización agregada del procesador, una señal que no distingue si el rendimiento está limitado por la capacidad aritmética del núcleo o por la transferencia de datos desde memoria; en el segundo caso, sostener la frecuencia máxima incrementa el consumo sin una mejora proporcional del tiempo de ejecución.

Este trabajo propone el diseño, la implementación y la evaluación de un agente en espacio de usuario que infiere el régimen de ejecución de la aplicación a partir de telemetría microarquitectónica y traduce esa inferencia en decisiones discretas de DVFS sobre CPU y GPU, con el objetivo de optimizar el Producto Energía–Retardo (EDP) sin incurrir en una penalización severa del rendimiento ni en una sobrecarga de inferencia que anule el beneficio

energético.

El desarrollo se organiza en cuatro fases: caracterización de cargas y construcción del conjunto de datos; entrenamiento y selección de un clasificador supervisado ligero; implementación del servicio de control; y validación experimental frente a los gobernadores nativos de Linux. El presente documento reporta el instrumento de medición y el protocolo experimental contruidos durante la primera fase — cuyo aporte metodológico central es un procedimiento reproducible para etiquetar como *compute-bound* o *memory-bound* cada ventana de ejecución de CPU o corrida completa de GPU, mediante una calibración empírica del modelo Roofline sobre la plataforma de medición, con trazabilidad explícita de la calidad de cada observación — junto con un primer entrenamiento y análisis del clasificador de GPU, posible ya dentro de esta fase por disponer de su conjunto de entrenamiento definido.

Abstract

Title: Design and Implementation of a User-Space Agent for Dynamic Frequency Management (DVFS) in Heterogeneous Systems through Lightweight Machine Learning Models.

Authors: Yeison Adrián Cáceres Torres; Ricardo Andrés Pérez Porras.

Keywords: DVFS; energy efficiency; high performance computing; Roofline model; micro-architectural telemetry; supervised machine learning.

Description:

Energy consumption has become a structural constraint on the scaling of high performance computing (HPC), particularly on heterogeneous nodes that combine CPUs and graphics accelerators. Dynamic Voltage and Frequency Scaling (DVFS) is the main control mechanism available at the hardware level, but exploiting it effectively requires deciding, at run time, which operating point suits the current workload behaviour. Native operating system governors base that decision on aggregate processor utilisation, a signal that does not distinguish whether performance is limited by the arithmetic capability of the core or by data transfer from memory; in the latter case, sustaining maximum frequency raises power draw without a proportional improvement in execution time.

This work proposes the design, implementation and evaluation of a user-space agent that infers the execution regime of an application from microarchitectural telemetry and translates that inference into discrete DVFS decisions over CPU and GPU, aiming to optimise the Energy-Delay Product (EDP) without incurring a severe performance penalty or an inference overhead that cancels the energy benefit.

The work is organised in four phases: workload characterisation and dataset construction; training and selection of a lightweight supervised classifier; implementation of the control

service; and experimental validation against the native Linux governors. This document reports the measurement instrument and experimental protocol built during the first phase, whose central methodological contribution is a reproducible procedure for labelling execution windows as *compute-bound* or *memory-bound* through an empirical calibration of the Roofline model on the measurement platform, with explicit traceability of the quality of each observation.

Índice general

Agradecimientos	1
Resumen	2
Abstract	4
Introducción	9
Planteamiento y justificación del problema	10
Objetivos	11
1. Marco de referencia	13
1.1. Marco Conceptual	13
1.1.1. Potencia CMOS y fundamento de DVFS	13
1.1.2. Modelo Roofline y regímenes de ejecución	14
1.1.3. Telemetría microarquitectónica con PMU	16
1.1.4. Espacios de ejecución <i>user-space</i> y <i>kernel-space</i>	20
1.1.5. Interfaces de observabilidad	21
1.1.6. P-states, gobernadores y DVFS	22
1.1.7. Cargas de calibración y modelos de clasificación ligera	24
1.1.8. Hiperparámetros, Optuna y validación	25
1.1.9. Producto Energía–Retardo (EDP)	26
1.2. Estado del Arte	27
1.2.1. Adaptación de la frecuencia según el comportamiento de la carga	28
1.2.2. Clasificación de cargas <i>compute-bound</i> y <i>memory-bound</i>	29
1.2.3. Actuación dinámica de frecuencia durante la ejecución	29
1.2.4. Brecha identificada y posición del trabajo	30
Repositorio y recursos complementarios	37

Índice de tablas

Índice de figuras

1.1. Modelo Roofline conceptual	15
1.2. Unidad de Monitorización de Rendimiento (PMU)	17
1.3. Contadores de núcleo y contadores de <i>uncore</i>	19
1.4. Espacios de usuario y de kernel	21
1.5. P-states, C-states y jerarquía DVFS	23

Introducción

En los sistemas de cómputo de alto rendimiento (HPC), el consumo energético se ha consolidado como una restricción crítica para el escalado sostenible, especialmente en arquitecturas heterogéneas CPU–GPU. En este contexto, el Escalado Dinámico de Voltaje y Frecuencia (DVFS) constituye el principal mecanismo de control a nivel de hardware, permitiendo ajustar los estados operativos de los procesadores para optimizar el compromiso entre consumo energético y tiempo de ejecución. Sin embargo, la efectividad de este mecanismo depende de la capacidad del sistema para adaptar dinámicamente la frecuencia al comportamiento cambiante de las aplicaciones, lo cual representa un desafío abierto en entornos HPC modernos.

En la literatura, este problema ha sido abordado principalmente mediante dos enfoques. Por un lado, los gobernadores de energía del sistema operativo, como *ondemand* o *schedutil*, emplean políticas reactivas basadas principalmente en la utilización del procesador, sin considerar la naturaleza microarquitectónica de la carga de trabajo, lo que limita su capacidad de adaptación a cargas de trabajo científicas dinámicas. Por otro lado, diversas propuestas han explorado estrategias heurísticas basadas en métricas microarquitectónicas, utilizando reglas deterministas para ajustar la frecuencia de operación. Aunque estos métodos presentan bajo *overhead*, su dependencia de umbrales estáticos y su limitada capacidad de generalización reducen su efectividad en escenarios donde las fases computacionales varían rápidamente.

A pesar de estos avances, persiste una limitación fundamental: la incapacidad de los enfoques actuales para capturar de forma robusta las transiciones dinámicas entre regímenes *compute-bound* y *memory-bound*, así como para adaptarse a la heterogeneidad CPU–GPU sin intervención manual o ajuste específico por carga de trabajo. Esta brecha evidencia la necesidad de mecanismos de control más flexibles que permitan inferir el régimen computacional de la aplicación en tiempo de ejecución y tomar decisiones de escalado de frecuencia fundamentadas en el perfil de ejecución.

En este contexto, el presente trabajo propone el diseño e implementación de un agente en

espacio de usuario basado en modelos ligeros de aprendizaje automático, capaz de clasificar en tiempo de ejecución las fases de una aplicación como compute-bound o memory-bound a partir de telemetría microarquitectónica, y de ajustar dinámicamente la frecuencia de CPU y GPU para optimizar el Producto Energía–Retardo (EDP). Se plantea demostrar que este enfoque mejora la eficiencia energética frente a los gobernadores nativos de Linux, manteniendo una degradación controlada del rendimiento y una sobrecarga de inferencia mínima y cuantificada mediante métricas experimentales.

Planteamiento y justificación del problema

Existe una ineficiencia energética estructural en la operación de los sistemas de alto rendimiento (HPC), especialmente en infraestructuras que integran aceleradores gráficos (GPU) para ejecutar aplicaciones científicas intensivas. Cuando una aplicación entra en una fase donde el procesamiento depende principalmente de la transferencia de datos desde memoria, mantener la CPU o la GPU a su frecuencia máxima no mejora el rendimiento de manera proporcional. En estas condiciones, los núcleos de cómputo permanecen parcialmente inactivos esperando datos, lo que genera un consumo energético elevado sin un beneficio equivalente en tiempo de ejecución. Esta desalineación entre demanda computacional y frecuencia operativa ha motivado el estudio del escalado de frecuencia como mecanismo para mejorar la eficiencia energética en aplicaciones científicas [35] y en sistemas de gran escala donde la gestión de potencia impacta el comportamiento global del sistema [10].

El problema se agrava por la naturaleza multifásica de las aplicaciones HPC, cuyo perfil de ejecución cambia a lo largo del tiempo de ejecución. En particular, estas aplicaciones alternan entre fases donde el rendimiento está limitado por la capacidad de cómputo del procesador (*compute-bound*) y fases donde está restringido por la transferencia de datos desde memoria (*memory-bound*). Esta distinción es crítica porque, en el régimen *memory-bound*, incrementos de frecuencia tienden a producir mejoras marginales en el tiempo de ejecución, mientras aumentan el consumo energético; en contraste, en fases *compute-bound* la frecuencia sí impacta el desempeño de forma más directa. Por ello, se han propuesto marcos de caracterización para identificar y clasificar estas fases en entornos HPC, con el fin de habilitar decisiones de control más informadas [4, 7].

A partir de este contexto, la gestión eficiente de frecuencia no depende únicamente de la disponibilidad del mecanismo DVFS en el hardware, sino de la capacidad del sistema para decidir, en tiempo de ejecución, cuál configuración resulta más conveniente según el comportamiento de la carga. Aunque se han propuesto mecanismos automatizados para seleccionar

frecuencias óptimas de GPU con el fin de mejorar la eficiencia energética sin comprometer significativamente el desempeño [2], persiste una brecha en la implementación de estrategias que integren de forma coordinada componentes CPU–GPU y que operen durante la ejecución de la aplicación, considerando además el costo computacional asociado al propio proceso de decisión.

Esta brecha resulta especialmente relevante en entornos académicos y científicos, donde el consumo eléctrico incide directamente en los costos operativos, la disponibilidad efectiva de recursos y la sostenibilidad institucional. En este contexto, contar con un mecanismo capaz de observar el comportamiento de la aplicación, inferir su fase de ejecución y ajustar la frecuencia de operación sin intervenir el código fuente constituye una alternativa con valor técnico y práctico.

Desde la perspectiva internacional, la literatura reciente ha mostrado avances en la optimización energética mediante escalado de frecuencia, gestión de potencia y caracterización de cargas HPC [35, 10, 4, 7, 2]. Sin embargo, a nivel nacional y regional, este tipo de soluciones aún no se encuentra ampliamente implementado en infraestructuras académicas de cómputo científico, particularmente bajo esquemas que combinen telemetría de bajo nivel, modelos ligeros de aprendizaje automático y control en espacio de usuario. En consecuencia, el problema no solo es técnicamente relevante, sino también pertinente en términos de aplicabilidad local.

A partir de esta necesidad de optimización energética en sistemas heterogéneos, se formula la siguiente pregunta de investigación que servirá como eje de desarrollo del proyecto:

¿En qué medida el diseño e implementación de un agente en espacio de usuario, guiado por modelos ligeros de aprendizaje automático, permite ajustar la frecuencia de operación (DVFS) en sistemas heterogéneos CPU–GPU para reducir el consumo energético, sin que el costo de la inferencia computacional degrade el rendimiento global de la aplicación, en comparación con los gobernadores nativos de Linux?

Objetivos

Objetivo General

Caracterizar, diseñar, implementar y evaluar un agente en espacio de usuario basado en modelos ligeros de Aprendizaje Automático para el ajuste dinámico de frecuencia (DVFS)

en sistemas heterogéneos CPU–GPU, con el propósito de maximizar la eficiencia energética en aplicaciones científicas sin inducir una penalización severa en su rendimiento global.

Objetivos Específicos

1. Caracterizar el comportamiento computacional y el consumo energético de cargas de trabajo representativas (intensivas en cómputo y en memoria) bajo distintos estados de frecuencia, recolectando telemetría de bajo nivel mediante contadores de rendimiento por hardware e interfaces de potencia estándar (Perf y RAPL para CPU y NVML para GPU).
2. Entrenar y validar modelos clásicos de Aprendizaje Automático (tales como Árboles de Decisión o Bosques Aleatorios) que sean capaces de clasificar, en tiempo de ejecución y con baja latencia de inferencia, las fases de ejecución de las aplicaciones basándose en la telemetría extraída.
3. Desarrollar un servicio de control (*daemon*) en el espacio de usuario que interactúe de forma asíncrona con el sistema operativo, leyendo el estado de los contadores de hardware, ejecutando la inferencia del modelo clasificador y aplicando políticas proactivas de DVFS a través de las interfaces estándar del sistema operativo, en función de la fase de ejecución inferida por el modelo.
4. Evaluar el impacto empírico del sistema propuesto mediante el cálculo del Producto Energía–Retardo (EDP), con el fin de determinar estadísticamente si el ahorro energético compensa la sobrecarga computacional (*overhead*) derivado de la inferencia del modelo en comparación con los gobernadores nativos de Linux.

Capítulo 1

Marco de referencia

1.1. Marco Conceptual

El ajuste dinámico de la frecuencia y del voltaje para reducir el consumo energético parte de una relación física entre el punto de operación del hardware, la potencia requerida y el tiempo de ejecución. Su efecto depende del régimen de la carga, porque una fase limitada por cómputo no responde de la misma forma que una fase condicionada por el acceso a memoria. En este trabajo, el régimen se define con el modelo Roofline y deja señales indirectas en la telemetría disponible; esas señales alimentan un clasificador ligero que debe inferir la clase con una latencia compatible con el lazo de control. La clase inferida se utiliza como contexto para una política DVFS, cuyo efecto se juzga mediante el Producto Energía–Retardo. Esta cadena separa la magnitud física empleada para construir la etiqueta de las señales accesibles durante la ejecución y delimita qué puede observarse, inferirse y controlarse en CPU y GPU.

1.1.1. Potencia CMOS y fundamento de DVFS

La potencia dinámica de un circuito CMOS se asocia con la conmutación de cargas capacitivas y depende del factor de actividad, de la capacitancia efectiva, del voltaje de alimentación y de la frecuencia. La relación utilizada como fundamento de DVFS se expresa como:

$$P_{\text{dyn}} = \alpha \cdot C \cdot V^2 \cdot f \quad (1.1)$$

donde α representa el factor de actividad, C la capacitancia efectiva conmutada, V el voltaje y f la frecuencia de operación [13]. El voltaje y la frecuencia no son variables

independientes en los estados de rendimiento, ya que los niveles de frecuencia más altos suelen requerir mayor voltaje para mantener las restricciones temporales del circuito. Por ello, una reducción de frecuencia que admita disminuir el voltaje puede reducir de forma apreciable la potencia dinámica debido a la dependencia cuadrática con V .

El beneficio energético no se deduce únicamente de esa reducción de potencia. La energía acumulada depende también del tiempo durante el cual se mantiene el consumo, de modo que una frecuencia menor puede resultar desfavorable si prolonga la ejecución en una proporción comparable o superior. La potencia estática permanece además como una contribución que DVFS no elimina, por lo que una ejecución más larga puede acumular un costo que reduzca el ahorro obtenido sobre la componente dinámica. La sensibilidad del tiempo a la frecuencia queda condicionada por el recurso que limita la fase. Una fase cuyo rendimiento depende de las unidades aritméticas suele responder con mayor intensidad a una reducción del reloj de cómputo, mientras que una fase condicionada por el tráfico de memoria puede conservar parte de su rendimiento al operar a menor frecuencia. Esta diferencia exige identificar el régimen antes de asociar una reducción de frecuencia con un beneficio energético.

1.1.2. Modelo Roofline y regímenes de ejecución

El modelo Roofline relaciona el rendimiento alcanzable con la intensidad operacional de la carga y con dos límites de la plataforma, el rendimiento aritmético pico y el ancho de banda pico de memoria. Su formulación básica es:

$$P = \min(P_{\text{pico}}, I \cdot BW_{\text{pico}}) \quad (1.2)$$

donde P_{pico} es el techo de cómputo, BW_{pico} el techo de ancho de banda e I la intensidad operacional [41]. Cuando el límite activo es $I \cdot BW_{\text{pico}}$, el rendimiento queda condicionado por el movimiento de datos y la carga se clasifica como *memory-bound*. Cuando el límite activo es P_{pico} , la capacidad aritmética domina y la carga se clasifica como *compute-bound*. La Figura 1.1 representa esta separación.

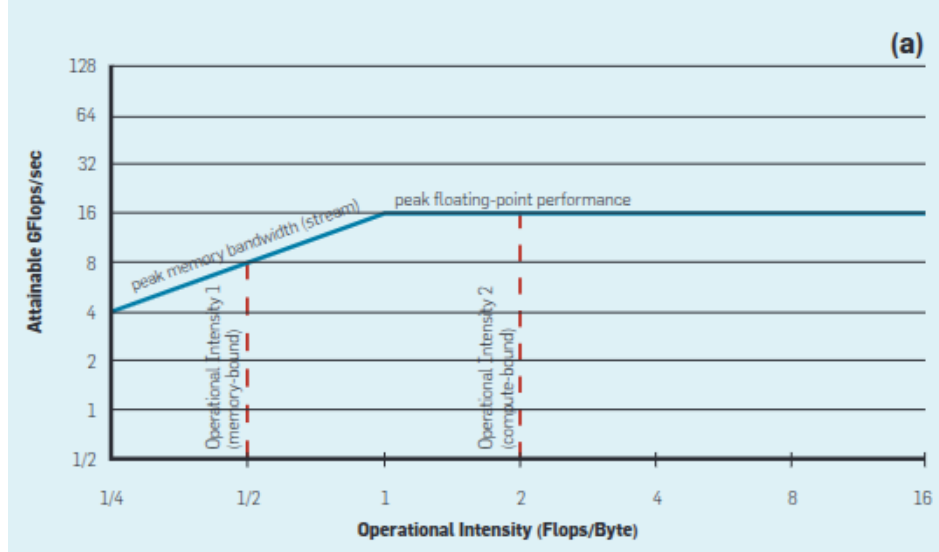


Figura 1.1: Modelo Roofline conceptual con el techo de ancho de banda de memoria y el techo de rendimiento de cómputo. Nota. Tomado de Williams et al. [41, Fig. 1(a)].

La intensidad operacional expresa operaciones por byte transferido desde el nivel de memoria considerado y, por ello, no equivale al tamaño nominal del conjunto de datos ni puede inferirse únicamente a partir de si este cabe en caché [41, p. 66]. La reutilización modifica el tráfico efectivo y puede elevar la intensidad incluso cuando el conjunto de trabajo completo excede la caché, como ocurre en implementaciones bloqueadas de multiplicación de matrices [14, Secs. 3–4]. Esta distinción evita usar propiedades estáticas del kernel como sustituto de una clasificación que depende del comportamiento medido sobre la plataforma. Roofline establece además un límite de rendimiento, no una predicción exacta del tiempo de ejecución. Una observación puede situarse por debajo de ambos techos por ineficiencias distintas del cuello de botella dominante, por lo que el uso del modelo en este trabajo se concentra en la separación de regímenes y en la construcción de una referencia física, sin interpretar la distancia al techo como una medida única de eficiencia.

La frontera entre ambos regímenes se encuentra en el punto de inflexión o *ridge point*, donde las dos ramas de la ecuación (1.2) se igualan. El umbral se obtiene como:

$$I_{\text{ridge}} = \frac{P_{\text{pico}}}{BW_{\text{pico}}} \quad (1.3)$$

Una intensidad inferior a I_{ridge} corresponde al régimen *memory-bound*, mientras que una intensidad superior corresponde al régimen *compute-bound* [41]. El punto de inflexión no representa una frecuencia óptima ni el máximo rendimiento de una aplicación. Marca la intensidad para la cual los dos límites del modelo tienen el mismo valor; una carga situada

cerca de esa frontera puede cambiar de régimen ante variaciones del techo aritmético, del ancho de banda o de su propio patrón de acceso. Su función en este trabajo es proporcionar un criterio físico de etiquetado, no una política de control por sí misma.

El valor del umbral debe corresponder a la precisión aritmética de las operaciones observadas, porque los techos de cómputo pueden diferir entre tipos de dato y precisiones [3]. También depende del punto de operación. El techo aritmético responde al reloj de las unidades de cómputo y el ancho de banda de memoria pertenece a otro dominio, por lo que ambos términos no tienen que variar en la misma proporción cuando cambia la frecuencia [16, 2]. En consecuencia, el *ridge point* utilizado para etiquetar una observación debe ser coherente con la precisión y con la configuración de frecuencia bajo la cual se obtuvo. Los valores nominales de la plataforma sirven como referencia, aunque la frontera empleada para clasificar observaciones debe corresponder a techos alcanzables bajo las condiciones de ejecución consideradas, razón por la que la caracterización experimental del Roofline precede al etiquetado [31].

La intensidad operacional observada se calcula sobre magnitudes que representan el mismo intervalo temporal. Para un intervalo i se utiliza:

$$I_i = \frac{\text{FLOPs}_i}{B_i^{\text{DRAM}}} \quad (1.4)$$

donde FLOPs_i representa el trabajo aritmético y B_i^{DRAM} los bytes transferidos hacia o desde memoria principal durante el mismo intervalo [41]. Los eventos PMU que se convierten en FLOPs dependen de la microarquitectura y su semántica debe verificarse antes de interpretar el conteo [40]. El tráfico DRAM se obtiene desde contadores asociados al controlador de memoria, cuyo alcance y granularidad pueden diferir de los contadores de núcleo. Esta diferencia obliga a alinear temporalmente ambos términos antes de aplicar la ecuación (1.4) y explica por qué la etiqueta Roofline se construye fuera del lazo ligero de inferencia.

1.1.3. Telemetría microarquitectónica con PMU

La clasificación en CPU requiere señales que describan el comportamiento de la ejecución sin reconstruir en línea la intensidad operacional. Las Unidades de Monitorización de Rendimiento, o PMU, proporcionan esa observabilidad mediante contadores de hardware que acumulan eventos como instrucciones retiradas, ciclos y actividad de la jerarquía de memoria. Los contadores pueden ser fijos o programables, y los segundos seleccionan eventos mediante códigos definidos por la arquitectura y calificadores específicos [19]. La Figura 1.2 resume esta organización. La PMU no interpreta el comportamiento de la aplicación, sino que entre-

ga conteos cuya semántica depende del evento y del dominio observado. Por ello, el mismo valor numérico solo adquiere significado después de considerar qué recurso cuenta, durante qué intervalo y con qué alcance. Los conteos funcionan como señales para la inferencia, pero no constituyen una medición directa de la intensidad operacional ni de la etiqueta Roofline.

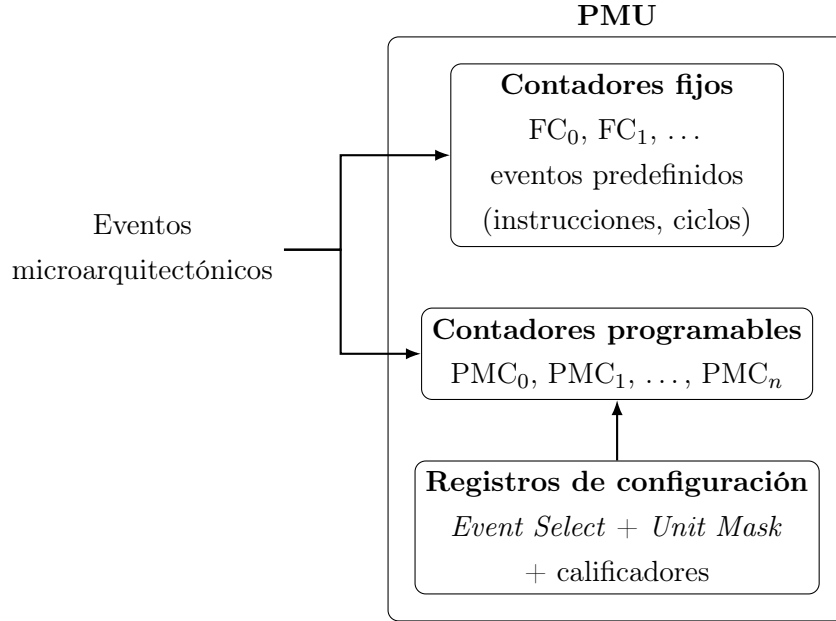


Figura 1.2: Organización conceptual de una Unidad de Monitorización de Rendimiento (PMU) con contadores fijos y programables. Nota. Elaboración propia.

La multiplexación aparece cuando se solicitan más eventos de los que pueden mantenerse simultáneamente en contadores físicos. En ese caso, el sistema alterna grupos de eventos y escala los conteos a partir del tiempo durante el cual cada grupo permaneció activo [29, 27]. El escalado conserva utilidad cuando el comportamiento es suficientemente estable durante la ventana, aunque puede introducir error si las fases cambian mientras un evento no está siendo contado. Esta limitación condiciona la cantidad de señales que conviene observar de forma simultánea.

Los eventos genéricos y los eventos crudos representan dos niveles de abstracción de la misma PMU. Los eventos genéricos ofrecen nombres comunes que el kernel traduce a cada procesador, mientras que los eventos crudos emplean directamente la codificación de la microarquitectura cuando se necesita una señal que la abstracción genérica no expone [39, 19]. La segunda opción exige verificar plataforma, código de evento, máscara y semántica, ya que una codificación válida en una microarquitectura no puede trasladarse automáticamente a otra. Esta diferencia también afecta la portabilidad del vector de características. Una señal expresada mediante un evento genérico puede mantener un contrato similar entre plataformas

aunque cambie su implementación, mientras que una señal cruda conserva un significado más específico y exige validar de nuevo la correspondencia física cuando cambia la microarquitectura. Para el clasificador, esa distinción delimita cuáles características pueden considerarse generales y cuáles quedan ligadas al procesador donde fueron medidas.

Los contadores de núcleo y los contadores de *uncore* difieren por su ámbito de atribución. Los primeros observan recursos asociados a un núcleo y pueden vincularse con la ejecución de una tarea, mientras que los de *uncore* cubren componentes compartidos del zócalo, entre ellos el controlador de memoria integrado y partes de la interconexión. Los eventos del controlador de memoria permiten observar el tráfico que alcanza DRAM, incluidos accesos de demanda y tráfico generado por mecanismos de precarga [18]. Esta señal resulta útil para el denominador de la intensidad operacional porque representa movimiento efectivo hacia memoria principal, pero su alcance de sistema impide atribuir el tráfico a un proceso individual sin condiciones de aislamiento adecuadas. La diferencia entre una medida local de núcleo y una medida compartida también explica por qué ambas fuentes pueden tener granularidades temporales distintas y no deben combinarse fila a fila sin una correspondencia temporal explícita. La Figura 1.3 muestra esta diferencia estructural.

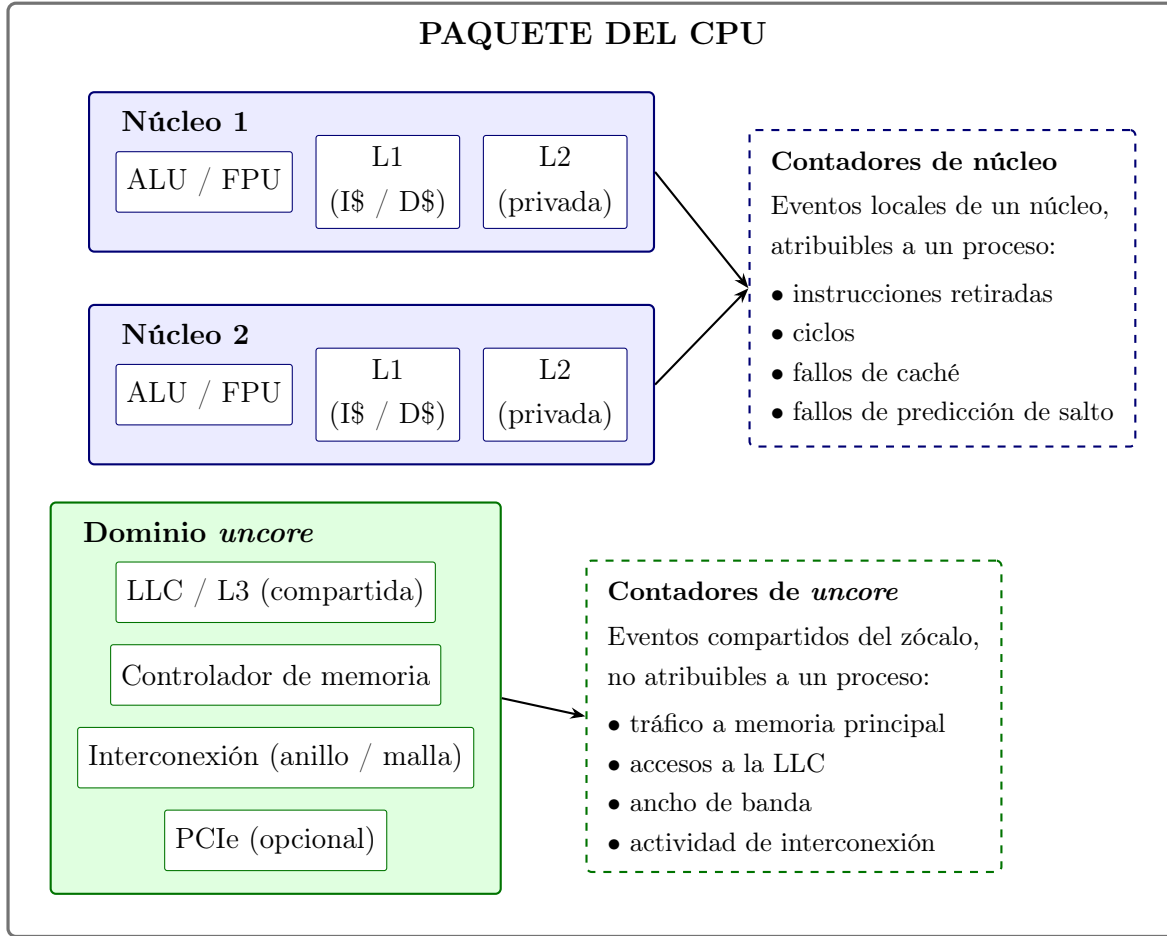


Figura 1.3: Ámbito de atribución de los contadores de núcleo y de *uncore* dentro del paquete del procesador. Los primeros observan eventos locales de un núcleo, mientras que los segundos observan recursos compartidos del zócalo. Nota. Elaboración propia.

A partir de los conteos pueden derivarse métricas compactas para describir la ejecución, entre ellas IPC y MPKI, sin que ninguna de ellas determine por sí sola si la carga es *compute-bound* o *memory-bound* [15, 27].

Una ventana de muestreo convierte los contadores acumulativos en una observación temporal mediante diferencias entre lecturas consecutivas. Su duración determina un compromiso entre resolución y sobrecarga, porque ventanas cortas localizan mejor los cambios de comportamiento, mientras que ventanas largas pueden mezclar fases distintas y reducen la frecuencia de lectura [27, 28]. Las tasas deben usar el tiempo realmente transcurrido entre lecturas y una referencia temporal monótonica [26]. La ventana define también la unidad temporal sobre la que el clasificador interpreta el estado. Si la fase cambia varias veces dentro del intervalo, la observación resume comportamientos distintos y la etiqueta pierde una correspondencia clara con un único régimen. Si el intervalo es demasiado corto, la medición

puede quedar dominada por ruido de conteo, costos de lectura o señales que se actualizan con menor frecuencia. Cuando una señal compartida se actualiza con menor granularidad que los contadores de núcleo, las magnitudes que se combinan deben agregarse al intervalo común antes de interpretar la observación.

La lectura de estas señales no depende únicamente del hardware disponible. El mecanismo que las consulta opera en un espacio de ejecución con permisos y fronteras definidas por el sistema operativo, condición que determina qué contadores e interfaces puede utilizar el agente.

1.1.4. Espacios de ejecución *user-space* y *kernel-space*

El agente se ubica en *user-space*, por lo que no accede directamente a los recursos reservados al kernel ni modifica sus estructuras internas. El sistema operativo media las operaciones sensibles y expone interfaces controladas para consultar contadores, leer energía o solicitar cambios de frecuencia [22, ch. 2]. Esta frontera, representada en la Figura 1.4, permite mantener el control fuera del kernel y al mismo tiempo impone restricciones de privilegio, alcance y latencia sobre las observaciones y actuaciones disponibles [30]. La arquitectura del proyecto depende, por tanto, de las interfaces que atraviesan esa frontera sin requerir instrumentación dentro del núcleo del sistema operativo.

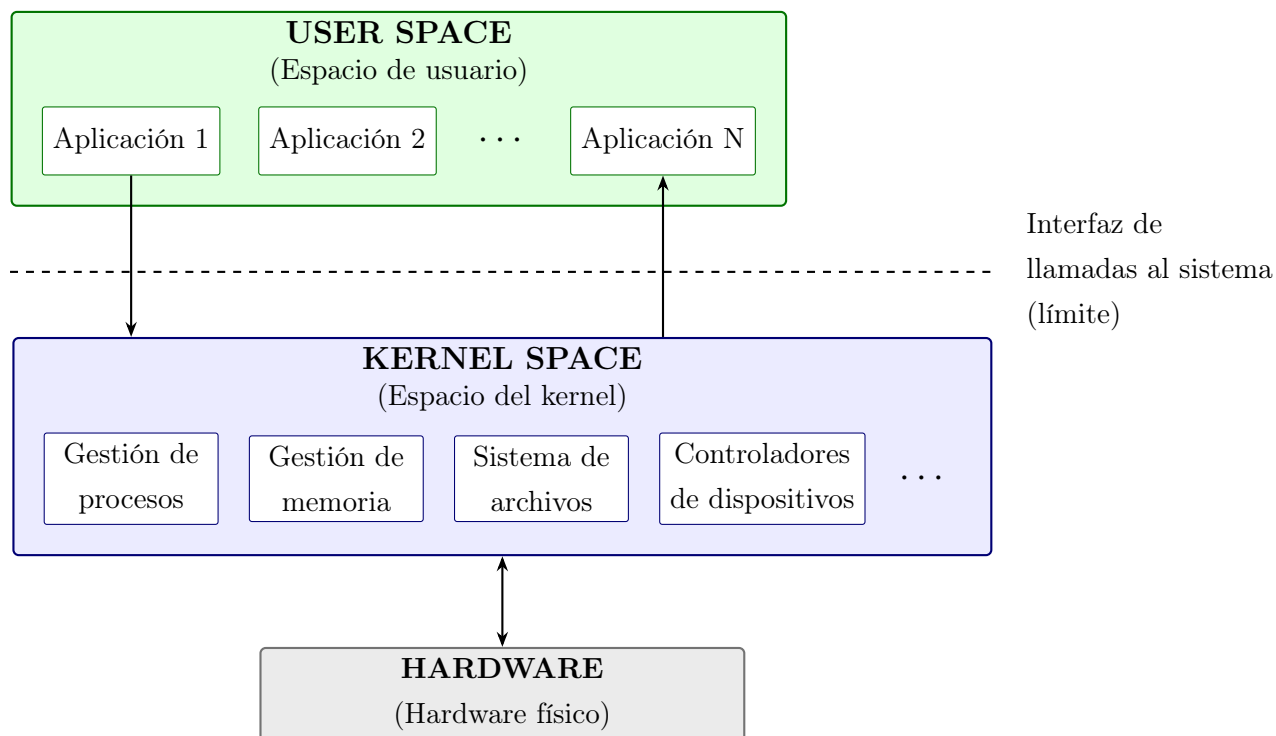


Figura 1.4: Relación conceptual entre *user-space*, *kernel-space* y el hardware físico. Nota. Elaboración propia.

1.1.5. Interfaces de observabilidad

Linux expone los contadores de CPU mediante `perf_event`, interfaz con la que el kernel configura eventos de la PMU, administra su asignación sobre los contadores físicos y devuelve los conteos a procesos en espacio de usuario [27]. Esta capa evita que el agente manipule directamente registros de la PMU y conserva las restricciones de alcance asociadas a cada evento. Un conteo puede asociarse a una tarea o a una CPU, y esa elección modifica qué actividad contribuye a la medida. En consecuencia, la interfaz no solo transporta un valor, también define el contexto de atribución con el que debe interpretarse. La disponibilidad efectiva depende tanto del soporte del procesador como de la política de acceso configurada en el sistema.

Intel RAPL proporciona contadores energéticos para los dominios que exponga la plataforma, entre ellos el paquete del procesador y, cuando está disponible, DRAM [11, 36]. Los valores se expresan como energía acumulada, de modo que el consumo de un intervalo se obtiene a partir de diferencias entre lecturas. La energía del dominio de paquete representa el consumo del paquete completo y no el consumo exclusivo de los núcleos reservados a la aplicación. Esta diferencia de alcance debe conservarse al interpretar el EDP, especialmen-

te cuando la medición de rendimiento se atribuye a una carga y la energía cubre recursos compartidos. El dominio DRAM, cuando existe, añade una medida separada de la energía asociada a memoria, pero tampoco identifica por sí mismo el régimen de ejecución. RAPL actúa así como fuente energética del sistema observado y no como contador microarquitectónico de la clase de régimen.

En GPU, NVML ofrece telemetría agregada del dispositivo, como utilización, potencia, energía y frecuencia, pero no entrega directamente los FLOPs y bytes requeridos por la intensidad operacional [32]. Sus métricas reflejan actividad y estado del acelerador durante ventanas internas del controlador, por lo que valores altos de utilización no bastan para distinguir si la limitación procede del cómputo o de la memoria. Nsight Compute accede a contadores de hardware con granularidad de kernel y puede obtener métricas necesarias para una caracterización Roofline más detallada, a costa de una instrumentación que puede requerir múltiples pasadas y una intrusión mayor [33]. En este marco, la diferencia relevante es de función conceptual. La caracterización detallada puede construir la referencia física fuera de línea, mientras que la inferencia durante la ejecución debe apoyarse en señales que puedan consultarse con baja intrusión. Esta asimetría entre la verdad de entrenamiento y la telemetría de producción forma parte de la limitación del problema, no de una equivalencia entre ambas fuentes. Observar el comportamiento resuelve solo una parte del lazo, porque la decisión resultante debe materializarse mediante los mecanismos de control de frecuencia de cada dispositivo.

1.1.6. P-states, gobernadores y DVFS

DVFS modifica el punto operativo del procesador mediante mecanismos de hardware y software que coordinan una solicitud de rendimiento con la actuación efectiva sobre el reloj y, cuando corresponde, el voltaje [17]. En CPU, los P-states representan niveles de rendimiento mientras el procesador permanece activo, mientras que los C-states representan niveles de inactividad con diferentes costos de reactivación. Los primeros modifican la capacidad disponible para una carga activa y los segundos reducen consumo cuando no existe trabajo suficiente para mantener el núcleo en actividad. Esta diferencia importa porque una política de frecuencia opera sobre capacidad de cómputo durante C0 y no sustituye a los mecanismos de reposo.

Sobre los P-states operan políticas de selección de rendimiento. Los gobernadores `performance` y `powersave` fijan solicitudes hacia extremos distintos del rango disponible, mientras que políticas reactivas como `ondemand` o `schedutil` ajustan el nivel solicitado a partir de señales administradas por el sistema operativo [24]. La política decide qué nivel pedir, pero el con-

trolador y el hardware determinan cómo esa solicitud se materializa. La decisión atraviesa el controlador, el firmware y la lógica de gestión del procesador antes de convertirse en una frecuencia efectiva, por lo que la frecuencia solicitada y la observada no tienen que coincidir en todo instante [17, 25]. Además, mecanismos autónomos del procesador pueden desplazar el reloj dentro de los límites permitidos según condiciones térmicas, de potencia y de actividad. Esta separación es necesaria al comparar una política propia con los gobernadores nativos, porque el objeto de la comparación es la decisión de control y no la suposición de que cada solicitud se cumple de manera instantánea.

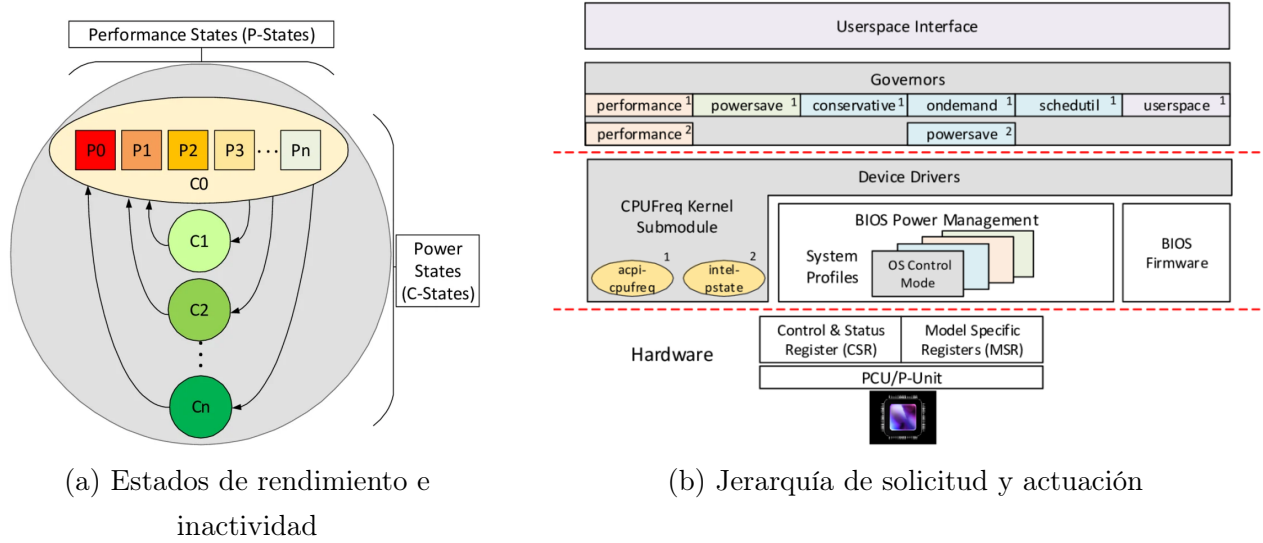


Figura 1.5: Relación entre P-states y C-states e integración de la solicitud de rendimiento desde espacio de usuario con los mecanismos que materializan el cambio en hardware. Nota. Tomado de Hebbar y Milenković [17].

El alcance de una solicitud de frecuencia depende de los dominios de control definidos por la plataforma. Con SMT habilitado, dos procesadores lógicos hermanos comparten un núcleo físico y una solicitud de mayor rendimiento de uno de ellos puede influir sobre la frecuencia efectiva observada por el otro [25]. Por ello, la afinidad de una carga y el dominio sobre el que actúa la política no son equivalentes, y la frecuencia efectiva debe interpretarse según la topología y el controlador activos [24].

En GPU, DVFS presenta dominios de actuación distintos, principalmente los relojes asociados al cómputo y a la memoria. La sensibilidad del tiempo de ejecución a cada dominio depende del comportamiento de la carga, por lo que una modificación del reloj de cómputo afecta con mayor intensidad a unas fases que a otras [16, 2]. Una carga limitada por el trabajo de los SM puede perder rendimiento al reducir ese reloj, mientras que una fase condicionada por el movimiento de datos puede mostrar una respuesta menor si el subsistema

de memoria continúa siendo el límite. Esta relación no implica que el reloj de memoria sea siempre independiente ni que exista una regla universal de frecuencia, porque el efecto depende de la arquitectura y de la aplicación. Además, el cambio de frecuencia tiene una latencia de actuación propia, de modo que una política que conmute con mayor rapidez que la respuesta del dispositivo puede introducir un costo comparable al beneficio esperado [37]. Estas restricciones hacen necesaria una señal de decisión que identifique el régimen sin repetir en línea la caracterización Roofline completa.

1.1.7. Cargas de calibración y modelos de clasificación ligera

Los kernels y benchmarks proporcionan ejecuciones acotadas sobre las que puede observarse el comportamiento computacional, mientras que los microbenchmarks aíslan propiedades de la plataforma como el techo aritmético o el ancho de banda de memoria [41, 15]. En este trabajo, las cargas de calibración establecen las referencias físicas del modelo Roofline y las cargas representativas generan observaciones cuya clase procede de esa referencia. La etiqueta no se deduce del nombre o de la reputación del kernel, porque el régimen depende del hardware, de la precisión y del punto operativo. El catálogo, las condiciones de ejecución y el procedimiento de etiquetado pertenecen a la metodología; en el marco interesa únicamente la relación entre una observación etiquetada y el modelo que aprende a inferirla.

El aprendizaje supervisado ligero se utiliza para aproximar esa relación mediante señales disponibles durante la ejecución. El calificativo *ligero* no designa una familia concreta, sino una restricción de uso. La inferencia debe introducir una latencia pequeña frente a la escala temporal en la que el agente decide, de manera que el costo del propio modelo no absorba el ahorro que pretende habilitar. También debe operar con características que puedan obtenerse en producción con una intrusión compatible con el mismo lazo. Un modelo entrenado con una señal que solo existe durante el perfilado offline podría alcanzar alta calidad predictiva y aun así no ser utilizable por el agente. La complejidad efectiva depende tanto de la familia como de sus hiperparámetros, por lo que la calidad predictiva se considera junto con el costo de inferencia y con la disponibilidad real de las características.

Las familias candidatas cubren fronteras de decisión con complejidades diferentes. La regresión logística funciona como referencia lineal de bajo costo y ofrece una base para determinar si una separación sencilla de las características es suficiente [20, Ch. 4]. Un árbol de decisión introduce fronteras no lineales e interacciones mediante reglas directamente inspeccionables, aunque una profundidad excesiva incrementa la varianza y el riesgo de ajuste al conjunto de entrenamiento [20, Ch. 8].

Random Forest combina árboles entrenados con muestreo y selección aleatoria de características, lo que reduce la dependencia de una única partición y suele adaptarse bien a conjuntos tabulares con relaciones no lineales [6]. Esta propiedad es pertinente cuando pocas señales microarquitectónicas interactúan de forma distinta según el régimen, porque la frontera puede depender de combinaciones de presión de memoria, actividad y frecuencia que no sean separables linealmente. Extra-Trees aumenta la aleatorización al considerar puntos de corte aleatorios y puede reducir el costo de construcción del ensamble, con un compromiso entre sesgo y varianza que depende de la configuración y de los datos [12]. Ambos modelos admiten pocas características de entrada y pueden representar interacciones que una frontera lineal no captura, aunque el número y la profundidad de los árboles incrementan el costo de inferencia. Un ensamble más flexible tampoco compensa señales poco informativas, por lo que la complejidad del modelo no sustituye la necesidad de una telemetría discriminante.

XGBoost construye árboles de forma secuencial para reducir una función objetivo regularizada y puede representar relaciones no lineales e interacciones complejas entre características [9]. Esa capacidad exige ajustar más decisiones de configuración y el costo del modelo depende del número y la profundidad de los árboles. Frente a los ensambles basados en árboles independientes, la construcción secuencial puede concentrar capacidad sobre errores residuales, aunque también aumenta la sensibilidad a la configuración y al tamaño efectivo del conjunto de entrenamiento. Ninguna de estas familias garantiza por sí misma una mejor clasificación. Con pocas características tabulares, un modelo simple puede generalizar mejor si la separación es estable, mientras que un ensamble puede resultar útil cuando la frontera depende de interacciones no lineales que se repiten entre familias de carga. Su utilidad depende de la información contenida en la telemetría y de la capacidad de generalizar hacia cargas que no participaron en el ajuste, lo que enlaza la selección del modelo con el tratamiento de sus hiperparámetros y con el esquema de validación.

1.1.8. Hiperparámetros, Optuna y validación

Un hiperparámetro controla la estructura del modelo o el procedimiento de ajuste y se fija fuera de la estimación ordinaria de sus parámetros. Los espacios de Random Forest, Extra-Trees y XGBoost combinan variables enteras, reales y decisiones con efectos no uniformes, por lo que una rejilla exhaustiva puede dedicar evaluaciones a combinaciones poco informativas. La búsqueda bayesiana basada en el estimador de Parzen estructurado, TPE, utiliza el historial de evaluaciones para proponer nuevas configuraciones [5]; Optuna implementa este tipo de optimización secuencial y permite explorar espacios mixtos sin enumerar todas las combinaciones [1]. El marco justifica el mecanismo de búsqueda, mientras que el

presupuesto de intentos y los rangos concretos pertenecen a la metodología y a los resultados.

La validación anidada separada por familia de kernel evita que la optimización de hiperparámetros utilice información de la familia reservada para estimar generalización. En cada partición externa, la búsqueda opera únicamente sobre el conjunto de entrenamiento y la familia externa se utiliza después para evaluar la configuración seleccionada, manteniendo la separación entre selección y estimación [8, 34]. Agrupar por familia es más exigente que repartir tamaños o variantes del mismo algoritmo entre ambos conjuntos, porque impide que patrones casi equivalentes aparezcan simultáneamente en entrenamiento y prueba. La misma restricción se aplica a observaciones correlacionadas que provienen de una unidad experimental común, porque distribuir las entre entrenamiento y prueba puede producir una estimación optimista. La validación intenta así medir generalización hacia patrones algorítmicos no observados, que es la condición relevante cuando el agente debe operar sobre cargas distintas de las utilizadas para construirlo.

La fuga de información también puede originarse en el vector de entrada. Si la etiqueta se construye comparando la intensidad operacional con el *ridge point*, incluir la propia intensidad, sus componentes directos o una variable que revele ese criterio permitiría al modelo reconstruir la etiqueta en lugar de inferirla a partir de señales disponibles en producción [21]. La magnitud que origina la etiqueta debe quedar fuera de las características de entrada. Esta separación conserva una diferencia explícita entre la fuente de verdad y los predictores. La primera puede requerir instrumentación costosa para construir etiquetas confiables, mientras que los segundos deben corresponder a señales que el agente pueda consultar durante una ejecución ordinaria. Bajo estas condiciones, la calidad predictiva y la latencia de inferencia siguen siendo criterios en tensión, porque un modelo más complejo puede mejorar la separación y al mismo tiempo consumir una fracción mayor del presupuesto temporal del lazo de control. El modelo seleccionado alimenta una política cuya utilidad final no se decide por la clasificación aislada, sino por su efecto conjunto sobre energía y tiempo.

1.1.9. Producto Energía–Retardo (EDP)

Una política DVFS puede reducir la potencia y aun así empeorar la eficiencia si el tiempo de ejecución aumenta lo suficiente. El Producto Energía–Retardo, o EDP, combina ambas magnitudes y se define como:

$$EDP = E \cdot T, \tag{1.5}$$

donde E es la energía consumida y T el tiempo de ejecución [23]. Un valor menor indica que, bajo este criterio, la reducción de energía compensa la variación temporal. La formulación puede ponderar con mayor intensidad el tiempo mediante:

$$ED^nP = E \cdot T^n, \quad (1.6)$$

donde n determina el peso asignado a la degradación temporal. Con $n = 1$ se recupera el EDP y con $n = 2$ se obtiene ED^2P [23, 2]. El exponente debe fijarse antes de comparar configuraciones, porque distintos valores pueden cambiar su orden relativo. El EDP tampoco sustituye el reporte separado de energía y tiempo. Dos configuraciones pueden producir valores próximos del producto mediante combinaciones diferentes de ambas magnitudes, de manera que el producto resume el compromiso pero no identifica por sí solo la causa del cambio. En este trabajo, el EDP proporciona el criterio conceptual para juzgar si una decisión de frecuencia produce un ahorro energético que justifique su costo temporal, en lugar de evaluar potencia o tiempo por separado.

El marco adoptado impone tres límites a esa interpretación. La clase de régimen es binaria y se define respecto a un umbral Roofline calibrado, por lo que no representa una escala continua de dependencia de cómputo o memoria ni identifica por sí sola el nivel de frecuencia más conveniente. La observabilidad es asimétrica entre CPU y GPU, ya que la CPU expone contadores microarquitectónicos mediante la PMU y la GPU ofrece en línea señales agregadas que actúan como proxies del comportamiento; por ello, una misma etiqueta puede tener distinta evidencia observable en cada dispositivo. Finalmente, toda actuación de frecuencia tiene un costo temporal que restringe la granularidad útil del control. Una clasificación correcta puede no traducirse en mejor EDP si el cambio de frecuencia llega tarde, si la fase es demasiado corta o si el punto seleccionado prolonga la ejecución. El EDP evalúa el resultado de la política bajo esas condiciones, sin convertir la clasificación o la telemetría disponible en una medición directa del beneficio energético.

1.2. Estado del Arte

La gestión energética en HPC ha pasado de aplicar configuraciones uniformes a explotar diferencias entre cargas y regiones de ejecución. DVFS ofrece el mecanismo de actuación, pero el beneficio depende de cómo la carga responde al cambio de frecuencia y de cuánto aumenta el tiempo cuando se reduce el punto operativo. Los antecedentes relevantes para este trabajo se concentran en tres problemas relacionados, la caracterización de esa sensibilidad,

la clasificación del régimen y la actuación dinámica durante la ejecución.

1.2.1. Adaptación de la frecuencia según el comportamiento de la carga

Calore et al. [7] analizaron DVFS sobre una CPU Intel Haswell y una GPU NVIDIA K80 mediante kernels y una aplicación HPC completa. Sus mediciones conjuntas de energía y tiempo mostraron que una única frecuencia no resulta igualmente conveniente para todas las regiones. La actuación a nivel de función permitió identificar oportunidades distintas según la región y la arquitectura, porque la reducción de reloj modifica de forma diferente el tiempo de secciones dominadas por cómputo y de aquellas cuya ejecución depende en mayor medida del movimiento de datos. El estudio no plantea un clasificador en línea, sino una adaptación apoyada en conocimiento de la estructura de la aplicación. Esa condición le permite actuar sobre regiones concretas, aunque exige conocer de antemano dónde se producen y en qué puntos se modificará la frecuencia. El aporte relevante para este trabajo es la evidencia de que la política de frecuencia debe responder al comportamiento de la carga y no únicamente a una configuración global mantenida durante toda la ejecución.

Guerreiro et al. [16] abordaron esa dependencia en GPGPU mediante métricas obtenidas en una configuración inicial. A partir de esa observación clasifican la sensibilidad de las aplicaciones frente a cambios en las frecuencias de cómputo y memoria, y utilizan la clase para anticipar su respuesta en otras configuraciones DVFS. El procedimiento evita ejecutar cada combinación posible y separa aplicaciones cuyo rendimiento responde principalmente al reloj de cómputo de aquellas más sensibles al dominio de memoria. Esta clasificación se aproxima al problema de inferir comportamiento a partir de telemetría, pero sus clases describen sensibilidad a los dominios de reloj y no una detección continua de fases *compute-bound* y *memory-bound*. El estudio se concentra además en GPU. La caracterización funciona como una etapa previa a la selección de configuración y supone que las métricas obtenidas inicialmente representan de forma suficiente el comportamiento posterior de la carga.

Ali et al. [2] también parten de métricas recolectadas en la configuración predeterminada de la GPU y construyen modelos de potencia y tiempo para estimar el comportamiento en otros estados DVFS. La selección considera consumo y degradación del desempeño para evitar que una reducción de potencia sea tratada como mejora cuando el tiempo crece en exceso. El enfoque es portable entre las plataformas evaluadas en la medida en que vuelve a construir la relación entre métricas y respuesta DVFS, en lugar de fijar una única frecuencia para todas las aplicaciones. En NVIDIA GA100 reportan una reducción promedio de energía de 29.6 % con una pérdida de rendimiento de 5.2 %, mientras que en NVIDIA GV100 obtienen 22.6 %

de reducción con una pérdida de 4.7 % [2]. Guerreiro et al. y Ali et al. reducen el costo de exploración mediante una caracterización previa y modelos que proyectan la respuesta en configuraciones no ejecutadas. La limitación frente a una aplicación multifásica es temporal, porque la caracterización inicial debe seguir representando la región sobre la que se aplica la decisión.

1.2.2. Clasificación de cargas *compute-bound* y *memory-bound*

Antici et al. [4] formulan con MCBound la distinción entre cargas *memory-bound* y *compute-bound* como un problema de clasificación basado en datos de ejecuciones HPC. La propuesta extrae características del comportamiento de los trabajos y aprende a separar ambos regímenes sin exigir que el usuario conozca previamente la naturaleza de cada aplicación. Sobre datos del supercomputador Fugaku, el método alcanza una puntuación F1-macro de al menos 0.89 y mantiene un costo reducido para el funcionamiento del sistema [4]. Este antecedente respalda la posibilidad de inferir automáticamente el régimen a partir de información observable, en lugar de asignarlo por conocimiento previo del nombre o del algoritmo. También muestra que la utilidad del clasificador debe evaluarse fuera de las observaciones utilizadas para ajustarlo, porque la pregunta relevante es si la separación se mantiene frente a trabajos que no participaron en el entrenamiento.

El alcance de MCBound es la caracterización de trabajos. La clase obtenida no se conecta directamente con un mecanismo que gobierne estados DVFS de CPU o GPU a medida que cambian las fases de una aplicación. Tampoco convierte la distinción entre regímenes en una regla de frecuencia concreta. Por ello, resuelve el problema de identificación bajo su contexto experimental, mientras deja separada la decisión de cómo convertir esa clase en una actuación de frecuencia, qué granularidad temporal debe utilizarse y cuánto cuesta realizar la inferencia dentro de un lazo de control. Esa separación coincide con la división que este trabajo establece entre construir una etiqueta físicamente sustentada, inferirla con telemetría ligera y evaluar después la política energética que la utiliza.

1.2.3. Actuación dinámica de frecuencia durante la ejecución

Simsek et al. [35] instrumentaron SPH-EXA para observar regiones de una simulación astrofísica y modificar la frecuencia de la GPU en puntos definidos de la ejecución. Frente a una frecuencia uniforme, reportan una reducción del consumo energético por GPU de hasta 7.82 % con una degradación del rendimiento de aproximadamente 2.95 % [35]. El resultado muestra que actuar dentro de la ejecución puede aprovechar diferencias entre regiones y confirma que la frecuencia conveniente puede variar durante una misma aplicación. La decisión,

sin embargo, queda ligada a la instrumentación del código y al conocimiento de las regiones sobre las que se debe intervenir. Una política de este tipo conoce el punto de cambio porque la aplicación fue preparada para exponerlo, mientras que un agente externo debe deducir el cambio a partir de señales observables.

Wang et al. [38] proponen DRLCAP, un esquema de *frequency capping* de GPU en línea basado en aprendizaje por refuerzo profundo. El sistema monitoriza información de ejecución, detecta cambios de fase y ajusta el límite de frecuencia, con una reducción promedio de 22 % en la energía de GPU y menos de 3 % de degradación de rendimiento en las arquitecturas NVIDIA evaluadas [38]. La propuesta integra observación y actuación dentro del mismo controlador y reduce la necesidad de definir manualmente una regla por región. A diferencia de una clasificación supervisada ligera seguida de una regla discreta, DRLCAP aprende la política mediante refuerzo profundo y se orienta exclusivamente a GPU. Su resultado aporta evidencia de que el control en línea puede producir ahorro, pero utiliza un mecanismo de decisión y un alcance de dispositivo distintos de los planteados en este trabajo. La diferencia impide trasladar directamente su arquitectura al problema CPU–GPU, aunque sirve como antecedente de un control que reacciona durante la ejecución.

1.2.4. Brecha identificada y posición del trabajo

Los antecedentes cubren por separado la adaptación de frecuencia según la sensibilidad de la carga, la clasificación del régimen y el control en línea. La integración de estas piezas sigue condicionada por el costo de observación, la latencia de decisión y las diferencias entre dispositivos. Los enfoques predictivos de GPU reducen el espacio de búsqueda a partir de una caracterización previa; MCBound aporta una clasificación explícita del régimen; los controladores dinámicos muestran que una decisión tomada dentro de la ejecución puede modificar el consumo. Ninguno de estos resultados implica por sí mismo que una única fuente de telemetría y una única política puedan trasladarse entre CPU y GPU. En CPU, los contadores PMU ofrecen señales microarquitectónicas de bajo nivel con alcances de núcleo y de *uncore*; en GPU, la telemetría ligera disponible durante la ejecución es más agregada y las mediciones Roofline de mayor detalle requieren herramientas de perfilado. Los mecanismos de actuación tampoco comparten la misma granularidad ni el mismo costo de transición.

El trabajo se sitúa en esa separación entre observación y actuación. La propuesta utiliza una etiqueta física común, *compute-bound* o *memory-bound*, pero admite que las señales con las que se intenta inferirla son distintas en CPU y GPU. El objetivo es integrar una clasificación de baja latencia con una política DVFS ejecutada desde espacio de usuario y evaluar su efecto mediante EDP frente a los mecanismos nativos de gestión de frecuencia.

Esta posición no supone que una misma regla de telemetría sea transferible entre dispositivos ni que la clasificación garantice ahorro energético; ambas condiciones deben evaluarse sobre la plataforma y las cargas utilizadas.

Bibliografía

- [1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2623–2631, 2019. doi: 10.1145/3292500.3330701.
- [2] G. Ali, M. Side, S. Bhalachandra, N. J. Wright, and Y. Chen. An automated and portable method for selecting an optimal gpu frequency. *Future Generation Computer Systems*, 149:71–88, 2023. doi: 10.1016/j.future.2023.07.011.
- [3] Oscar Antepara, Leonid Oliker, and Samuel Williams. Roofline analysis of tightly-coupled CPU-GPU superchips: A study on MI300A and GH200. In *Proceedings of the SC ’25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1205–1216, 2025. doi: 10.1145/3731599.3767497.
- [4] F. Antici, A. Bartolini, Z. Kiziltan, O. Babaoglu, and Y. Kodama. Mcbound: An online framework to characterize and classify memory/compute-bound hpc jobs. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15, Atlanta, GA, USA, 2024. doi: 10.1109/SC41406.2024.00062.
- [5] James S. Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. Algorithms for hyper-parameter optimization. In *Advances in Neural Information Processing Systems*, volume 24, pages 2546–2554, 2011.
- [6] Leo Breiman. Random forests. *Machine Learning*, 45:5–32, 2001. doi: 10.1023/A:1010933404324.
- [7] E. Calore, A. Gabbana, S. F. Schifano, and R. Tripiccione. Evaluation of dvfs techniques on modern hpc processors and accelerators for energy-aware applications. *Concurrency and Computation: Practice and Experience*, 29(12):e4143, 2017. doi: 10.1002/cpe.4143.

- [8] Gavin C. Cawley and Nicola L. C. Talbot. On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, 11:2079–2107, 2010.
- [9] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. doi: 10.1145/2939672.2939785.
- [10] M. T. Costa et al. Characterizing the impact of gpu power management on an exascale system. In *Proc. SC '25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC Workshops '25)*, pages 1524–1533, New York, NY, USA, 2025. doi: 10.1145/3731599.3767702.
- [11] H. David, E. Gorbato, U. R. Hanebutte, R. Khanna, and C. Le. RAPL: Memory power estimation and capping. In *Proc. 16th ACM/IEEE International Symposium on Low Power Electronics and Design (ISLPED)*, pages 189–194, 2010. doi: 10.1145/1840845.1840883.
- [12] Pierre Geurts, Damien Ernst, and Louis Wehenkel. Extremely randomized trees. *Machine Learning*, 63(1):3–42, 2006. doi: 10.1007/s10994-006-6226-1.
- [13] R. Gonzalez, B. M. Gordon, and M. A. Horowitz. Supply and threshold voltage scaling for low power CMOS. *IEEE Journal of Solid-State Circuits*, 32(8):1210–1216, 1997. doi: 10.1109/4.604018.
- [14] Kazushige Goto and Robert A. van de Geijn. Anatomy of high-performance matrix multiplication. *ACM Transactions on Mathematical Software*, 34(3):1–25, 2008. doi: 10.1145/1356052.1356053. Article 12.
- [15] B. Gregg. *Systems Performance: Enterprise and the Cloud*. Prentice Hall, Upper Saddle River, NJ, USA, 2nd edition, 2020.
- [16] J. Guerreiro, A. Ilic, N. Roma, and P. Tomás. DVFS-aware application classification to improve GPGPUs energy efficiency. *Parallel Computing*, 83:93–117, 2019. doi: 10.1016/j.parco.2018.02.001.
- [17] R. Hebbar and A. Milenković. PMU-events-driven DVFS techniques for improving energy efficiency of modern processors. *ACM Transactions on Modeling and Performance Evaluation of Computing Systems*, 7(1):1–31, 2022. doi: 10.1145/3538645.

- [18] Intel Corporation. *3rd Gen Intel Xeon Processor Scalable Family, Codename Ice Lake, Uncore Performance Monitoring Reference Manual*, May 2021. URL <https://cdrdv2-public.intel.com/679093/639778%20ICX%20UPG%20v1.pdf>. Revision 1.0.
- [19] Intel Corporation. Intel 64 and ia-32 architectures software developer’s manual, volume 3b: System programming guide, part 2 (performance monitoring, nmi watchdog interaction with fixed-function and general-purpose counters), 2026. URL <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>. Accessed: Aug. 12, 2026.
- [20] Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani. *An Introduction to Statistical Learning: with Applications in R*. Springer Texts in Statistics. Springer, 2nd edition, 2021. doi: 10.1007/978-1-0716-1418-1.
- [21] Shachar Kaufman, Saharon Rosset, Claudia Perlich, and Ori Stitelman. Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4):1–21, 2012. doi: 10.1145/2382577.2382579.
- [22] M. Kerrisk. *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. No Starch Press, San Francisco, CA, USA, 2010.
- [23] J. H. Laros III, K. Pedretti, S. M. Kelly, W. Shu, K. Ferreira, J. Van Dyke, and C. Vaughan. Energy delay product. In *Energy-Efficient High Performance Computing: Measurement and Tuning*, SpringerBriefs in Computer Science, chapter 6, pages 51–55. Springer, London, UK, 2013. doi: 10.1007/978-1-4471-4492-2_9.
- [24] Linux Kernel Documentation. CPU Performance Scaling, 2026. URL <https://docs.kernel.org/admin-guide/pm/cpufreq.html>. Accessed: Sep. 5, 2026.
- [25] Linux Kernel Documentation. intel_pstate cpu performance scaling driver, 2026. URL https://docs.kernel.org/admin-guide/pm/intel_pstate.html. Accessed: Sep. 5, 2026.
- [26] Linux man-pages project. clock_gettime(3) — linux manual page, 2026. URL https://man7.org/linux/man-pages/man3/clock_gettime.3.html. Accessed: Aug. 29, 2026.
- [27] Linux man-pages project. perf_event_open(2) — linux manual page, 2026. URL https://man7.org/linux/man-pages/man2/perf_event_open.2.html. Accessed: Aug. 29, 2026.

- [28] Linux perf project. perf-stat(1) — linux manual page, 2026. URL <https://man7.org/linux/man-pages/man1/perf-stat.1.html>. Accessed: Aug. 29, 2026.
- [29] T.-Y. Liu, J. Guo, and B. Huang. Efficient cross-platform multiplexing of hardware performance counters via adaptive grouping. *ACM Transactions on Architecture and Code Optimization*, 21(1):1–26, 2024. doi: 10.1145/3629525.
- [30] S. McCarty. Architecting containers part 1: Why understanding user space vs. kernel space matters. Red Hat Blog, 2015. URL <https://www.redhat.com/en/blog/architecting-containers-part-1-why-understanding-user-space-vs-kernel-space-matters>. Accessed: Apr. 3, 2025.
- [31] National Energy Research Scientific Computing Center (NERSC). Roofline performance model. NERSC Documentation, 2026. URL <https://docs.nersc.gov/tools/performance/roofline/>. Accessed: Aug. 7, 2026.
- [32] NVIDIA. Nvidia management library (nvml). NVIDIA Developer Documentation, 2026. URL <https://docs.nvidia.com/deploy/nvml-api/index.html>. Accessed: Apr. 3, 2026.
- [33] NVIDIA. Nsight compute. NVIDIA Developer Documentation, 2026. URL <https://docs.nvidia.com/nsight-compute/>. Accessed: Aug. 8, 2026.
- [34] David R. Roberts, Volker Bahn, Simone Ciuti, Mark S. Boyce, Jane Elith, Gurutzeta Guillera-Aroita, Severin Hauenstein, José J. Lahoz-Monfort, Boris Schröder, Wilfried Thuiller, David I. Warton, Brendan A. Wintle, Florian Hartig, and Carsten F. Dormann. Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure. *Ecography*, 40(8):913–929, 2017. doi: 10.1111/ecog.02881.
- [35] O. S. Simsek, J.-G. Piccinali, and F. M. Ciorba. Increasing energy efficiency of astrophysics simulations through gpu frequency scaling. In *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1826–1834, Atlanta, GA, USA, 2024. doi: 10.1109/SCW63240.2024.00229.
- [36] P. Thamm and U. Leser. Strategies to measure energy consumption using RAPL during workflow execution on commodity clusters. *arXiv preprint arXiv:2505.09375*, 2025.
- [37] D. Velicka, O. Vysocky, and L. Riha. Methodology for GPU frequency switching latency measurement. In *Proc. 2025 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pages 830–839, 2025. doi: 10.1109/IPDPSW66978.2025.00133.

- [38] Y. Wang, M. Hao, H. He, W. Zhang, Q. Tang, X. Sun, and Z. Wang. DRLCap: Runtime GPU frequency capping with deep reinforcement learning. *IEEE Transactions on Sustainable Computing*, 9(5):712–726, 2024. doi: 10.1109/TSUSC.2024.3361596.
- [39] V. M. Weaver. Self-monitoring overhead of the linux perf event performance counter interface. In *Proc. IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 102–111, 2015.
- [40] V. M. Weaver and S. A. McKee. Can hardware performance counters be trusted? In *Proc. IEEE International Symposium on Workload Characterization (IISWC)*, pages 141–150, Seattle, WA, USA, 2008. doi: 10.1109/IISWC.2008.4636099.
- [41] S. Williams, A. Waterman, and D. Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.

Repositorio y recursos complementarios

Repositorio público del proyecto:

<https://github.com/AdrianCCRS/hyperion>